INFO20003: Database Systems

Lecturer: Egemen Tanin

Lecture 22

Special Data Types: A Case Study

Week 12

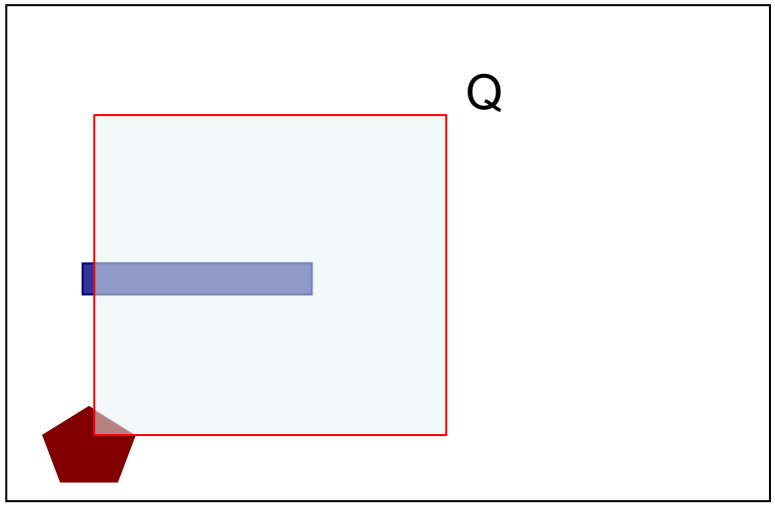
DBs and Special Data Types

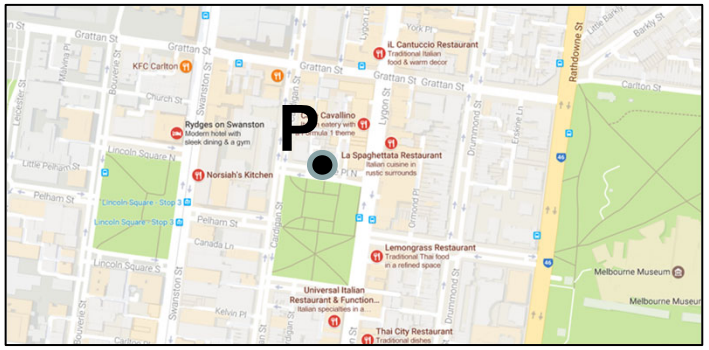
- Unlike data that we can process using string comparisons or hashing based on numerical ids, or other common methods we saw in earlier lectures, there are many other types of data that exist that are harder to process, retrieve and do many other operations with
- They are increasingly in use in DBs and that requires special attention
- We see more and more images, videos, audio etc in DBs
- The key issue in general is how to query with these data types and in particular how to query with high efficiency
- We study one data type in this lecture to give an example

Spatial Data

For example, spatial data requires more complex computations for accessing data, e.g., intersections of objects in space to find overlaps

There is no trivial way to sort items which is a key issue in many cases, e.g., below is a common range query on a simple set of two items in space and a nearest neighbor query





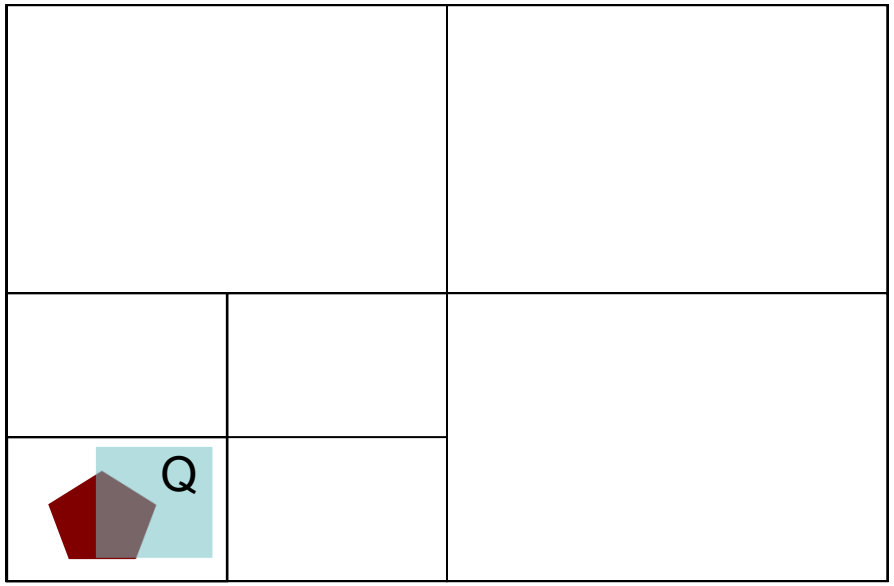
Novel Ways to Index Data

Indexing special data types effectively is a key means to process queries efficiently in DBs

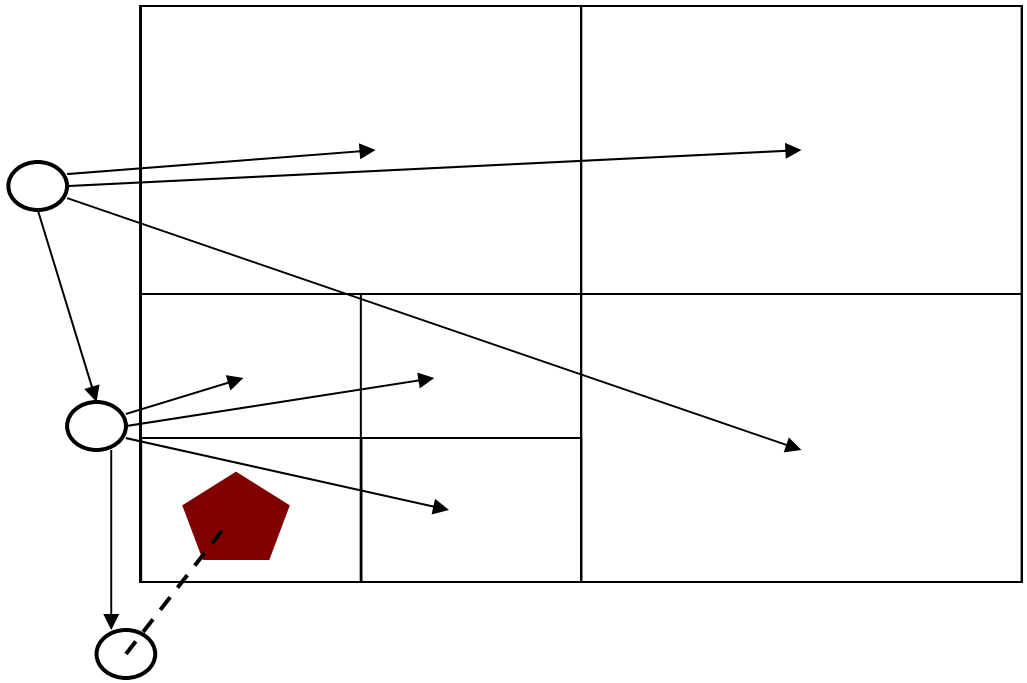
For spatial data in 2-d space, for exponentially reducing the number of comparisons/retrievals, many classes of novel indices were invented

They are in use in Oracle Spatial and other comparable DBMS extensions

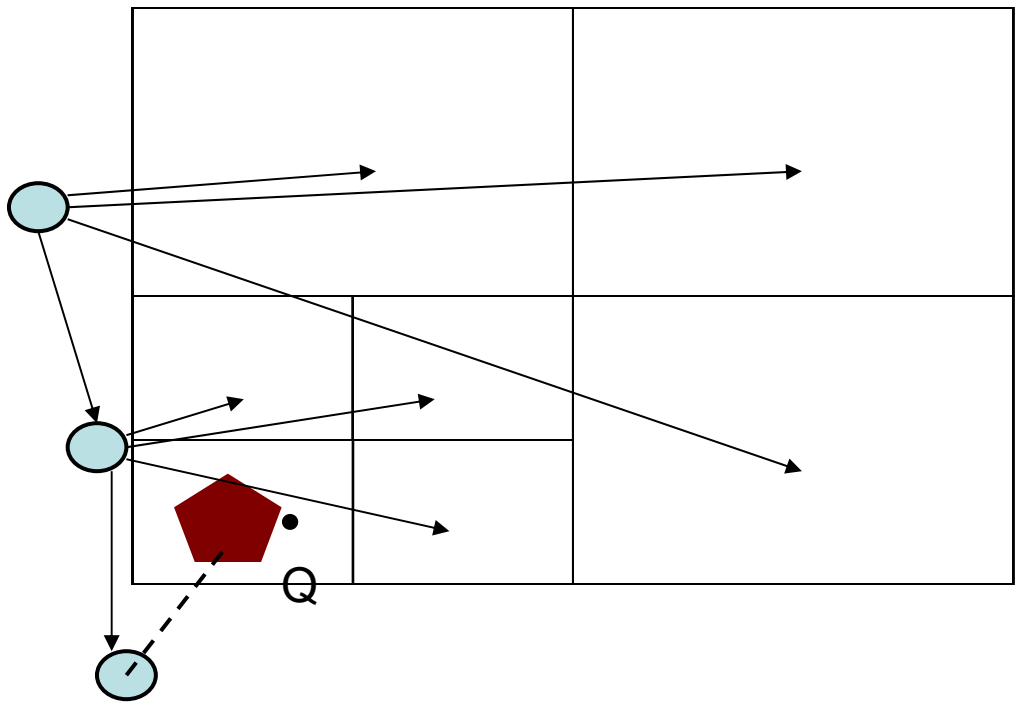
The following class of indices is one such: called Quadtrees



But where is the tree structure?



How to run nearest neighbor query

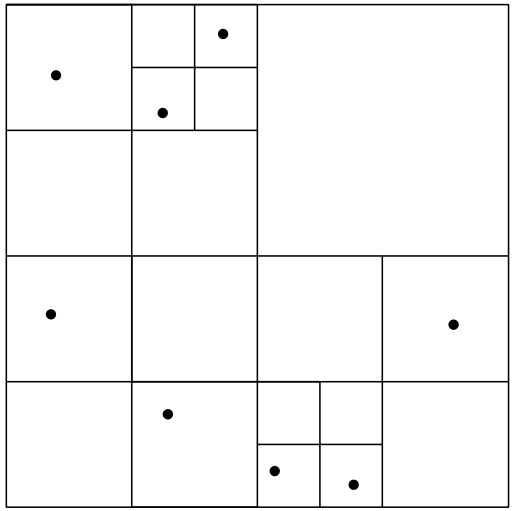


Basically via similar approach that we do in other trees we saw before

Just order the access with respect to distance to a point in this case

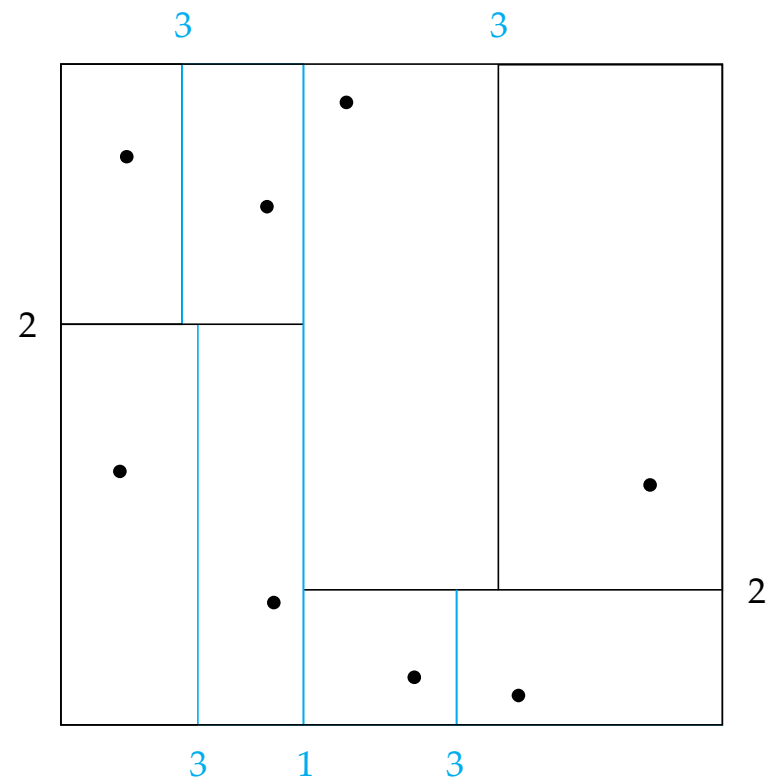
Versions of Quadtrees

- Each node of a quadtree is associated with a rectangular region of space; the top node is associated with the entire target space.
- Each division happens with respect to a rule based on data type.
- Each non-leaf node divides its region into four equal sized quadrants.
- Thus each such node has four child nodes corresponding to the four quadrants and division continues recursively until a stopping condition.
- Example: **<u>Leaf nodes have between zero and some fixed maximum number of points (set to 1 in example below)</u>**



Many other index types exist here

- **k-d tree** – another structure used for indexing in multiple dimensions.
- **Each level of a *k-d* tree partitions the space into two.**
  - Choose one dimension for partitioning at the root level of the tree.
  - Choose another dimension for partitioning in nodes at the next level and so on, cycling through the dimensions.
- In each node, *approximately half of the points stored in the sub-tree fall on one side and half on the other.*
- Partitioning stops when a node has less than a given number of points.

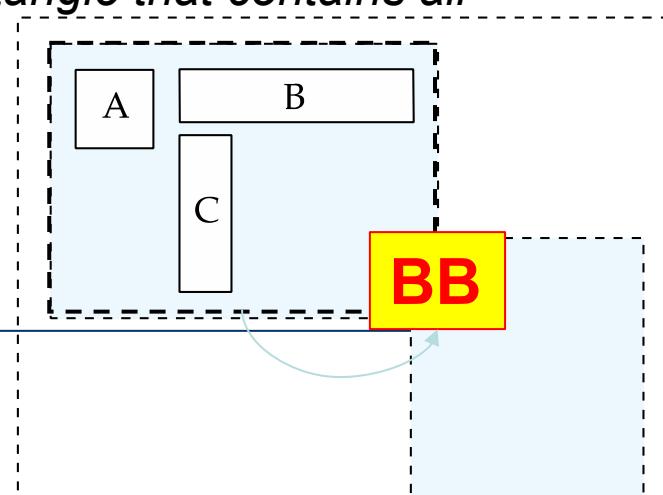


R-Trees

- **R-trees** can be considered as a **N-dimensional version of B^+-trees**, useful for indexing sets of rectangles and other polygonal data.
- **Supported in many modern database systems**, with variants like R^+ -trees and R*-trees.
- **Basic idea:** generalize the notion of a one-dimensional interval associated with each B+ -tree node to an **N-dimensional interval**, that is, an N-dimensional rectangle.
- **We will consider only the common two-dimensional case** (N = 2)
  - generalization for $N > 2$ is straightforward, **although R-trees work well only for relatively small N**

Disadvantage: Bounding boxes of children of a node are allowed to overlap

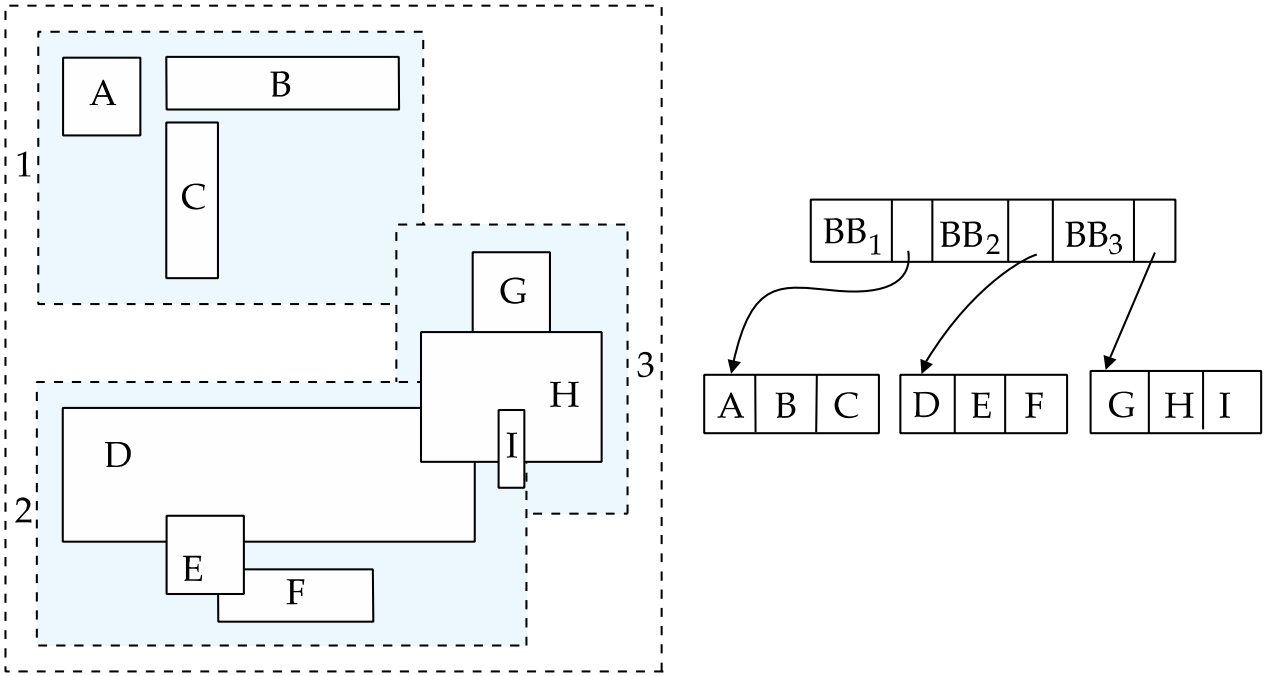
- *A **bounding box** (BB) of a node is a minimum-sized rectangle that contains all the rectangles/polygons associated with the node*



© University of Melbourne

Example R-Tree

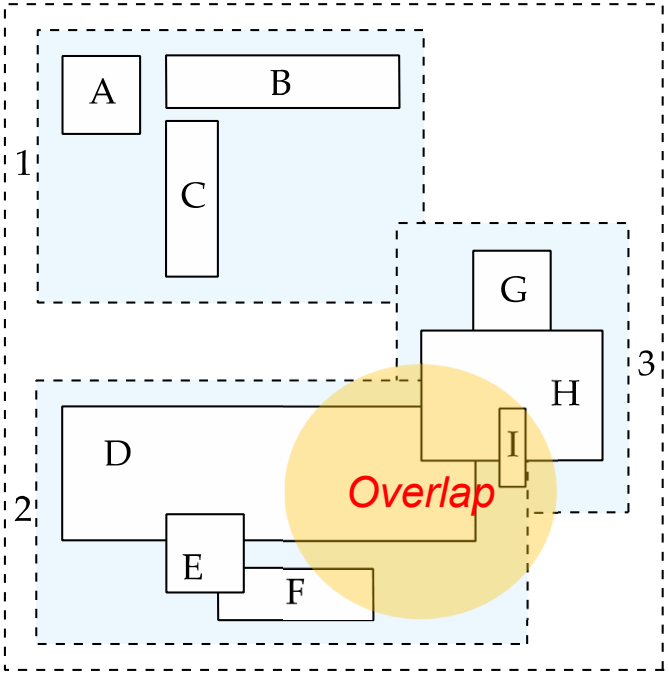
- A set of rectangles (solid line) and the bounding boxes (dashed line) of the nodes of an R-tree for the rectangles.
- The **R-tree is shown on the right**.
- The clustering of objects are based on a rule.



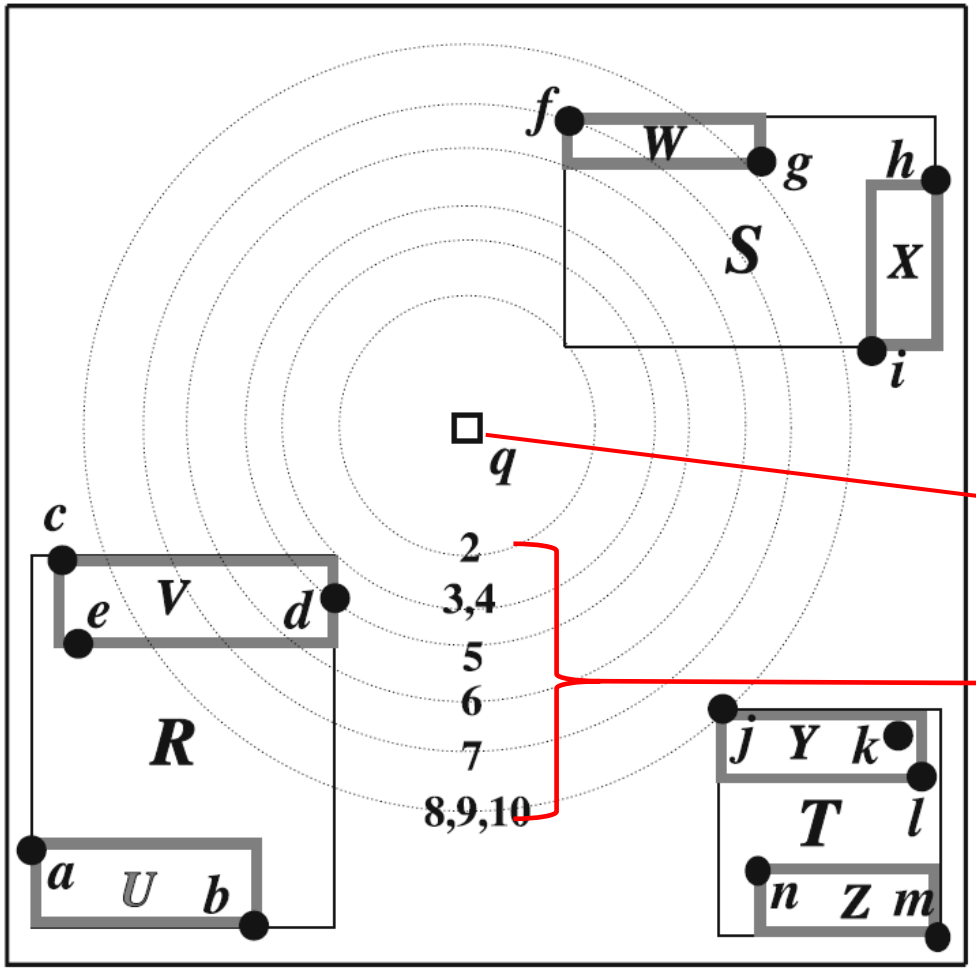
Search in R-Trees

- To find data items intersecting a given query point/region, do the following, starting from the root node:
  - **If the node is a leaf node, output the data items whose keys intersect** the given query point/region.
  - Else, **for each child of the current node whose bounding box intersects the query point/region, recursively search** the child.

- Can be very inefficient in worst case **since multiple paths may need to be searched due to overlaps**, but works fine in practice.



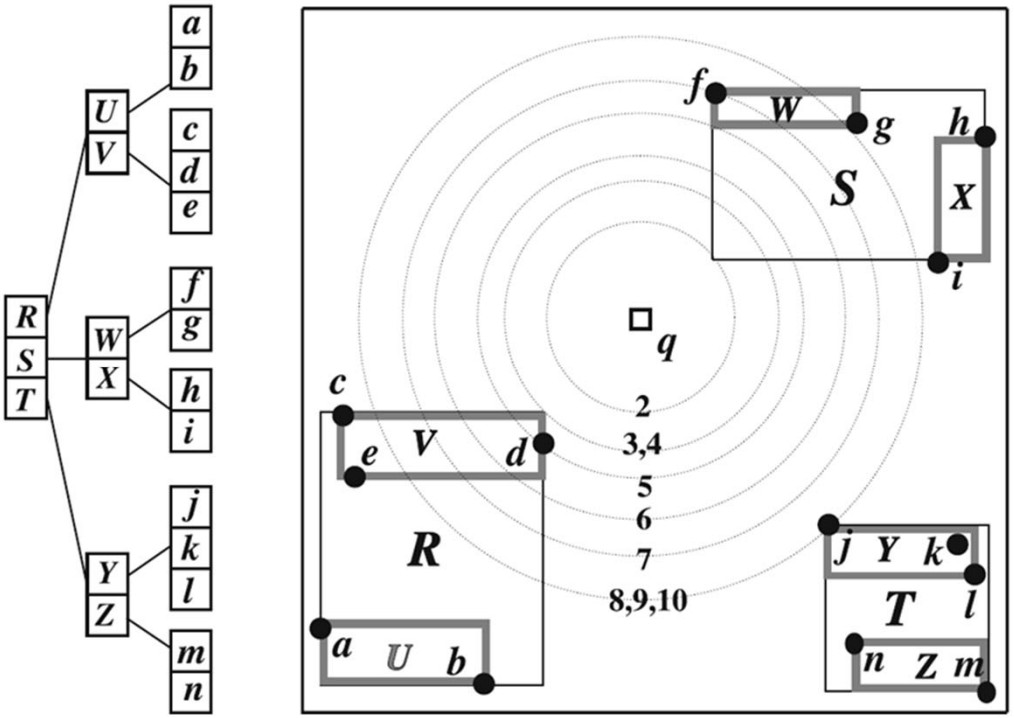
Another Example R-tree



Query Point for NN

Order of BBs or items wrt distances

Nearest Neighbor Query on R-tree



Step	Priority queue	Retrieved NN(s)
1	$\langle S, R, T \rangle$	$\langle\rangle$
2	$\langle R, W, T, X \rangle$	$\langle\rangle$
3	$\langle V, W, T, X, U \rangle$	$\langle\rangle$
4	$\langle \boldsymbol{d}, W, T, X, \boldsymbol{c}, \boldsymbol{e}, U \rangle$	$\langle\rangle$

Step 5 finds d as the first NN (using Best First search)

What to do now?

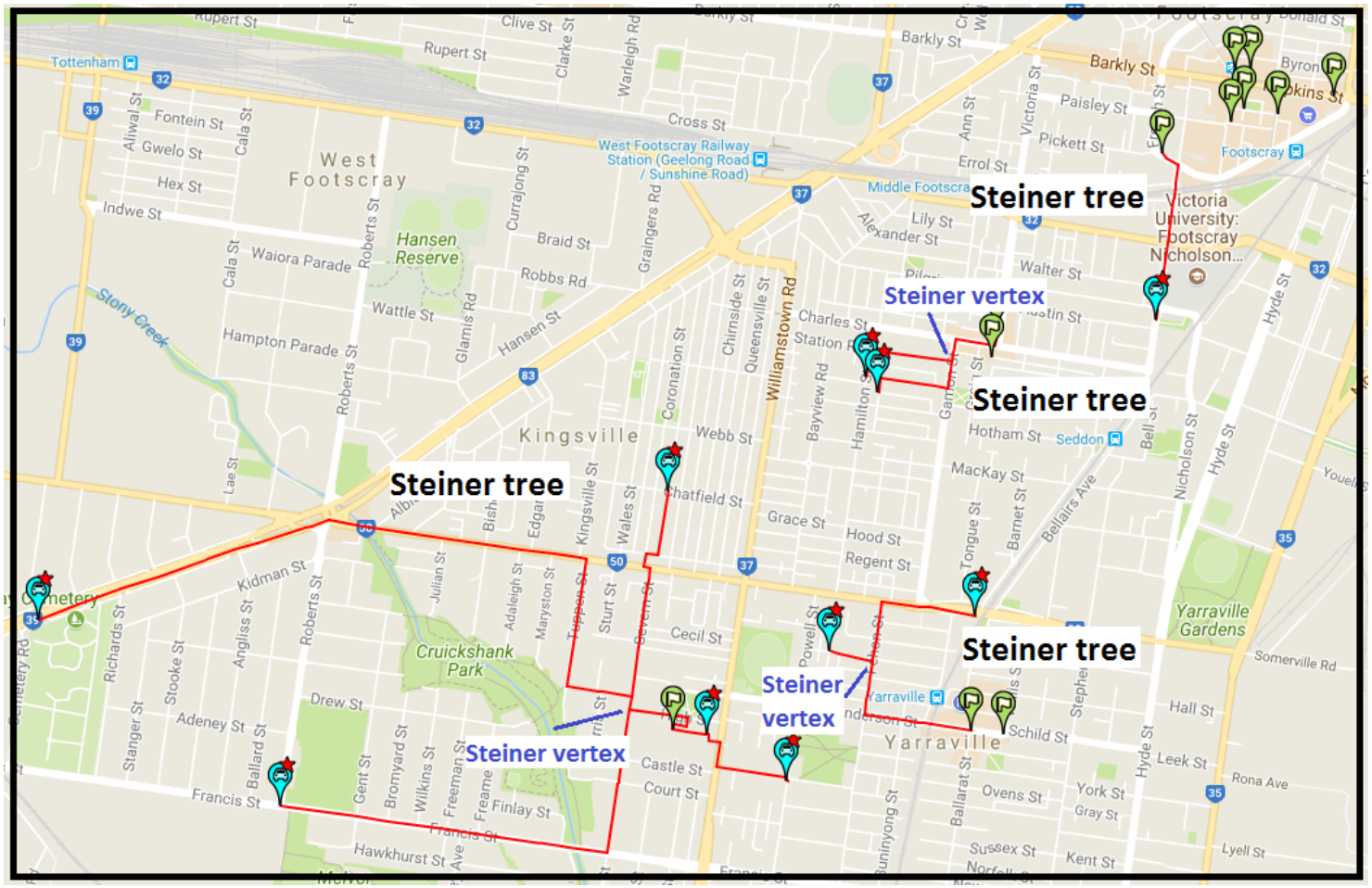
- **There is no point going through all index types** for all data types
  - There is hundreds of them
  - Even each type would have many subtypes
    - E.g., MX-CIF quadtree is one quadtree type among many
  - Same with other indices
- Given a data set, when uploading to the DBMS
  - Find **the potential query types**
  - Research what **indices that particular DBMS would have for that data type**
  - Research **what queries you would better do on what index**
  - **Create index** mainly when you have large data
  - **Monitor performance**
  - **Tune or create other indices**
  - Your DMBS will have a version of the **create index** command

© University of Melbourne

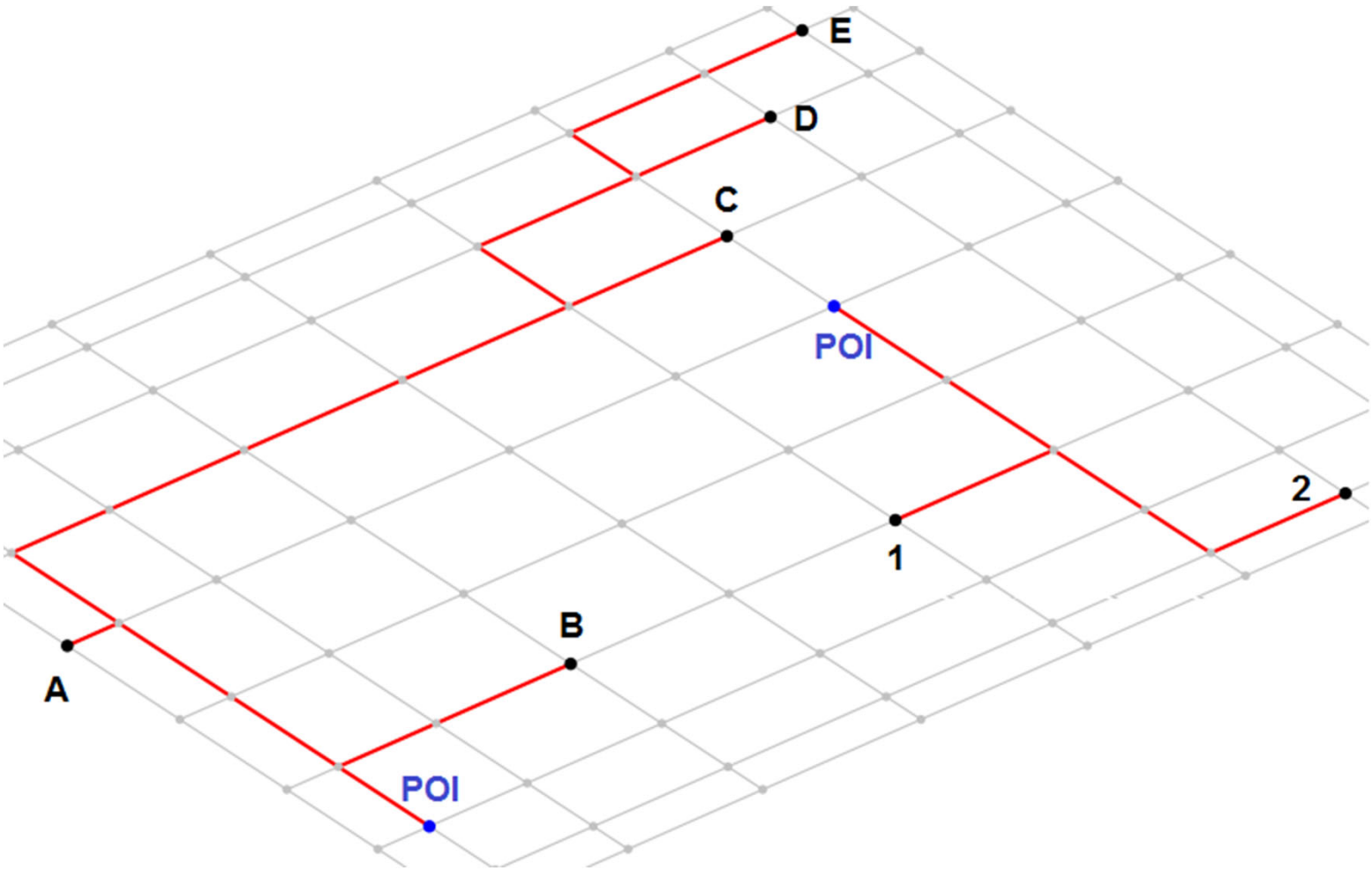
Indexing is one, algorithms next

- For special data types and queries DBMS would implement extensions that have special algorithms
- For example: if you need to do shortest path computations, your DBMS would these days commonly have one internally
- You run these commonly using extensions to SQL
- Today's DBMS come with shortest path, traveling salesman etc problems and associated algorithms to solve them implemented

Cutting edge example…



A simplified version…



Extensions

- Many databases today come with extensions to deal with special data

- For example: Oracle Spatial

https://www.oracle.com/au/database/spatial/

© University of Melbourne